人物画像可视化项目企业化升级八周计划

- 人物画像可视化项目企业化升级八周计划
  - 文档说明
  - 一、先说结论：推荐的最终方向
    - 1. 推荐技术路线
    - 2. 为什么推荐这一套
    - 3. 为什么不建议继续只用原生 HTML 迭代
  - 二、你当前项目的现实状态评估
    - 1. 你已经做到了什么
    - 2. 你当前的主要问题不在“low”，而在“工程化不够”
  - 三、Vue、React、Next.js、Nuxt 到底该怎么选
    - 1. Vue 能不能用
    - 2. React 能不能用
    - 3. 我的建议：选 React，但不是裸 React
    - 4. Vue 路线什么时候也值得选
  - 四、推荐技术栈分别是做什么的
    - 1. Next.js
    - 2. React
    - 3. TypeScript
    - 4. FastAPI
    - 5. PostgreSQL
    - 6. SQLAlchemy
    - 7. Alembic
    - 8. Auth.js / 登录鉴权方案
    - 9. Playwright
    - 10. GitHub Actions
    - 11. Docker
    - 12. Sentry
  - 五、企业化升级的底层逻辑图
    - 1. 不只是“换框架”，而是升级四个层次
    - 2. 你的项目升级思路图
    - 3. 你以后要具备的完整工程图
  - 六、八周总体安排总览
    - 八周总目标图
    - 八周完成后的目标状态
  - 七、八周详细计划
  - 第 1 周：定路线、搭脚手架、补核心基础
    - 本周目标
    - 为什么先做这个
    - 本周学习内容
    - 本周要做的项目任务

    - 本周交付物
    - 本周验收标准
  - 第 2 周：组件化前端基础与设计系统初版
    - 本周目标
    - 本周学习内容
    - 本周要做的项目任务
    - 本周交付物
    - 本周验收标准
  - 第 3 周：后端模块化改造
    - 本周目标
    - 本周学习内容
    - 本周要做的项目任务
    - 本周交付物
    - 本周验收标准
  - 第 4 周：数据库设计与数据入库
    - 本周目标
    - 本周学习内容
    - 本周要做的项目任务
    - 本周交付物
    - 本周验收标准
  - 第 5 周：迁移群体页与学生页
    - 本周目标
    - 本周学习内容
    - 本周要做的项目任务
    - 本周交付物
    - 本周验收标准
  - 第 6 周：迁移对标模块与资源模块
    - 本周目标
    - 本周学习内容
    - 本周要做的项目任务
    - 本周交付物
    - 本周验收标准
  - 第 7 周：权限、测试、监控与质量保障
    - 本周目标
    - 本周学习内容
    - 本周要做的项目任务
    - 本周交付物
    - 本周验收标准
  - 第 8 周：收尾上线、文档沉淀、下一阶段规划
    - 本周目标
    - 本周学习内容
    - 本周要做的项目任务
    - 本周交付物
    - 本周验收标准

- 八、学习顺序建议
  - 推荐优先级
- 九、你后面最需要形成的能力，不是背 API，而是这几种思维
  - 1. 组件思维
  - 2. 数据思维
  - 3. 分层思维
  - 4. 可维护思维
  - 5. 系统思维
- 十、最容易踩的坑
  - 坑 1：一上来就全量重写
  - 坑 2：只换前端，不动数据
  - 坑 3：同时学太多技术
  - 坑 4：太早追求复杂架构
  - 坑 5：忽略文档和测试
- 十一、八周之外的中长期规划
  - 1. 后台内容维护系统
  - 2. 数据审核机制
  - 3. 报告导出
  - 4. 更正式的权限系统
  - 5. 更稳定的生产部署
- 十二、给你的最终建议
- 十三、推荐官方学习资料
  - 前端与框架
  - 后端与数据库
  - 工程与质量
  - 鉴权
- 十四、最后的落地建议
- 附录：一句话版本的技术定位
  - 当前项目
  - 目标项目
  - 核心升级路线

人物画像可视化项目企业化升级八周计划

文档说明

这份文档面向当前“人物画像可视化”项目，目标不是泛泛讨论前端框架，也不是只给一个技术名词列表，而是要回答下面这些更实际的问题：

- 这个项目如果要向企业级网站/系统靠拢，应该往什么方向升级？
- 到底是继续堆原生 HTML，还是迁移到 Vue / React 一类框架？
- 如果迁移，应该迁到什么技术栈，为什么？
- 未来 8 周具体应该怎么安排，先做什么，后做什么？

- 每周应该学习哪些技术，它们分别解决什么问题，怎么使用？
- 什么是“企业化升级”的底层逻辑，而不是只换个壳？

本计划结合当前项目现状来设计：你已经有页面、有后端、有数据、有线上部署，所以最好的路线不是推倒重来，而是“保留业务逻辑，升级工程承载方式”。

一、先说结论：推荐的最终方向

1. 推荐技术路线

对于你当前这个项目，最推荐的企业化升级路线是：

```
前端：Next.js + React + TypeScript
后端：FastAPI
数据库：PostgreSQL
ORM：SQLAlchemy
数据库迁移：Alembic
鉴权：Auth.js（或基于 JWT 的登录方案）
图表：ECharts
测试：pytest + Playwright
部署：Docker + GitHub Actions + Render / 云服务器
监控：Sentry
```

2. 为什么推荐这一套

原因不是“React 比 Vue 高级”，也不是“流行就该用”，而是因为你的项目类型更像：

- 数据驱动平台
- 教学管理/分析系统
- 多页面、多图表、多权限、多角色系统
- 后期会有后台录入、数据维护、日志、导出和 AI 能力

这类项目更适合“前后端分层、数据正式化、页面组件化”的路线，而不是继续在单个 HTML 中堆 JavaScript。

3. 为什么不建议继续只用原生 HTML 迭代

原生 HTML + 内嵌 JS 的优势是：

- 起步快
- 页面立刻可见
- 原型开发很顺手

但它的劣势会随着项目增长迅速放大：

- 页面一多，重复代码越来越多
- 同类组件无法统一维护

- 接口请求、状态管理、页面逻辑容易散落各处
- 后续加登录、权限、后台管理时很痛苦
- 改一个公共样式或公共逻辑，容易牵一发而动全身

所以，企业化升级的第一步不是“再加一页”，而是“让项目可持续维护”。

二、你当前项目的现实状态评估

1. 你已经做到了什么

当前项目并不是普通网页作业，而是一个功能完整的原型系统，已经具备以下能力：

- 有多个页面组成完整业务链路
- 有群体画像页和个人画像页
- 有高校对标模块
- 有资源承接页和方法说明页
- 有 FastAPI 后端
- 有 CSV 数据源与前端 JS 数据源
- 有 AI 问答接口
- 有 GitHub + Render 线上访问能力

换句话说，你现在最大的优势不是技术栈，而是**业务逻辑已经很完整**。

2. 你当前的主要问题不在“low”，而在“工程化不够”

当前项目距离企业级系统的差距，主要不在于颜色、图表或者页面看起来够不够高级，而在于下面这些方面：

- 数据还是 CSV / JS 文件，缺少正式数据库
- 前端逻辑直接写在 HTML 内部，维护成本高
- 没有统一的组件层和接口层
- 没有登录、权限、后台维护系统
- 没有操作日志、异常监控、测试体系
- 没有明确的数据更新和审核流程

这也是为什么后面的升级重点不应该是“换个视觉风格”，而应该是“升级系统骨架”。

三、Vue、React、Next.js、Nuxt 到底该怎么选

1. Vue 能不能用

可以，当然可以。Vue 适合：

- 入门更平滑

- 单人开发体验好
- 模板语法直观
- 中小型后台项目也非常常见

如果你已经熟悉 Vue，走 `Vue + Nuxt` 也是一条完全可行的路线。

2. React 能不能用

也可以，而且更适合你当前这个项目未来的发展方向。React 的优势在于：

- 组件化思维更彻底
- 配套生态成熟
- 企业平台和中后台场景很常见
- 与 Next.js 结合后，适合同时做管理页面和内容页面
- TypeScript 配合体验成熟

3. 我的建议：选 React，但不是裸 React

推荐路线不是：

```
React + 自己拼路由 + 自己拼构建 + 自己拼 SSR
```

而是：

```
Next.js + React + TypeScript
```

原因很简单：你需要的是一套更完整的工程框架，而不是只要一个前端库。

4. Vue 路线什么时候也值得选

如果你满足下面任意一种情况，可以考虑 Vue：

- 你自己已经比较熟 Vue
- 你更看重短期开发速度
- 你做的是偏内容站，而不是偏系统平台
- 你后面合作团队以 Vue 为主

但从你现在这个项目的走向看，我仍然更建议：

Next.js + React + TypeScript

四、推荐技术栈分别是做什么的

这一节不只告诉你名字，还告诉你“它到底解决什么问题”。

1. Next.js

它是什么

Next.js 是 React 的全栈框架，负责：

- 页面路由
- 服务端渲染 / 静态生成
- 项目构建
- 前端工程组织
- API 路由（如果需要）
- 部署适配

在你项目中的作用

它会成为你未来新的前端工程骨架，把现在这些独立 HTML 页面变成：

- 群体总览页
- 个人画像页
- 高校对标总览页
- 多校比较页
- 单校详情页
- 资源页
- 方法页

全部统一纳入同一个可维护的前端项目中。

你怎么用它

你会主要用到：

- `app/` 目录做页面路由
- `layout.tsx` 做公共布局
- `page.tsx` 做具体页面
- 组件目录 `components/`
- 数据请求层 `services/` 或 `lib/`
- 类型定义 `types/`

学它的目标

不是学全部，而是学会：

- 页面怎么拆
- 路由怎么建
- 公共布局怎么做
- 页面怎么调用后端接口
- 组件怎么复用

---

2. React

它是什么

React 是一个组件化 UI 库，用来把页面拆成可复用单元。

在你项目中的作用

它会把你现在分散在 HTML 中的大块结构，拆成：

- 顶部导航组件
- KPI 卡片组件
- 图表容器组件
- 风险标签组件
- 学生列表组件
- 对标项目卡片组件
- 比较表组件
- AI 对话面板组件

你怎么用它

你会主要接触：

- `props`
- `state`
- `useEffect`
- 表单状态
- 条件渲染
- 列表渲染
- 组件拆分

学它的目标

学会“页面不是一整块 HTML，而是多个小组件拼出来的”这个思维。

---

3. TypeScript

它是什么

TypeScript 是 JavaScript 的类型增强版。

在你项目中的作用

它会约束你的接口返回、组件 props、表格字段、图表数据结构，减少后期维护时的混乱。

举例说，现在后端返回的学生对象包含：

- `student_id`
- `name`
- `warning_level`
- `student_type`

以后在前端你会给它定义类型，而不是靠“猜字段名”。

为什么它重要

当项目变大后，类型就是稳定性的基础。企业项目里，接口一多，TypeScript 的价值会非常明显。

---

4. FastAPI

它是什么

FastAPI 是 Python Web 框架。

在你项目中的作用

它是你现有后端逻辑的承载者。后面不建议换掉，因为你现在的优势就在于：

- 学生画像逻辑已经在 Python 里
- 数据处理已经在 Python 里
- AI 调用也已经在 Python 里

你后面怎么用它

要做的不是“学会 FastAPI 基础语法”这么简单，而是把你现在的后端从“单文件脚本”升级成“模块化服务”：

- `api/` 存接口
- `models/` 存数据库模型
- `schemas/` 存请求和响应结构
- `services/` 存业务逻辑
- `core/` 存配置、数据库连接、权限等基础能力

---

5. PostgreSQL

它是什么

正式关系型数据库。

在你项目中的作用

它将替代：

- `formal_student_data.csv`
- `assets/js/benchmark_data.js`

成为正式数据存储层。

为什么一定要上数据库

因为 CSV 和 JS 文件适合 demo，但不适合企业系统：

- 不方便多人维护
- 不方便查询和筛选
- 不方便增删改查
- 不方便做权限和日志
- 不方便做审核状态

未来适合放进数据库的数据

- 学生基本信息
- 学生维度得分
- 风险标签
- 建议路径
- 高校对标项目数据
- 校内资源导航数据
- 操作日志
- 用户与角色信息

---

6. SQLAlchemy

它是什么

Python ORM，用于把数据库表映射成 Python 对象。

在你项目中的作用

未来你不再直接手写所有 SQL，而是通过模型类去组织数据结构。

例如：

- `Student`
- `StudentDimension`
- `BenchmarkProject`

- `ResourceItem`

学它的目标

学会下面这些能力就够用：

- 建表模型
- 基础查询
- 条件筛选
- 分页
- 关联查询
- 插入和更新

---

7. Alembic

它是什么

数据库迁移工具。

在你项目中的作用

当数据库表结构变化时，你不能靠手改数据库；而应该通过迁移脚本来管理版本变化。

这在企业项目里非常关键，因为数据库结构不是一次性定死的。

你怎么用它

典型流程是：

```
修改 SQLAlchemy 模型
  -> 生成迁移脚本
  -> 执行迁移
  -> 数据库结构同步更新
```

---

8. Auth.js / 登录鉴权方案

它是什么

登录与会话管理方案。

在你项目中的作用

未来这个系统一旦用于更正式场景，就不可能所有人都直接看全部内容，至少要区分：

- 管理员

- 导师 / 老师
- 普通查看者

你要学什么

- 登录是什么
- token / session 是什么
- 路由保护是什么
- 权限控制是什么

你前期不一定非得马上上完整 SSO，但要先把权限模型想清楚。

---

9. Playwright

它是什么

前端端到端测试工具。

在你项目中的作用

它能自动帮你验证这些事：

- 页面能不能打开
- 登录能不能走通
- 学生列表能不能点开详情
- 对标比较页能不能正常显示图表

为什么它重要

你的项目一旦从原型升级到正式工程，测试是必须补上的，不然每次改页面都容易破坏原有功能。

---

10. GitHub Actions

它是什么

自动化工作流。

在你项目中的作用

可以让项目在 push 代码后自动执行：

- 前端 lint
- 前端测试
- 后端测试

- 构建
- 部署

企业化意义

它让项目不是靠“手动检查”，而是逐步拥有自动质量保障。

---

11. Docker

它是什么

容器化工具。

在你项目中的作用

它能把你的前端、后端、数据库环境标准化，解决“我电脑能跑，你电脑不能跑”的问题。

为什么后期重要

项目越正式，环境一致性越重要。Docker 不一定第一周学，但后期一定要补。

---

12. Sentry

它是什么

错误监控平台。

在你项目中的作用

如果线上页面报错、接口异常、用户看到白屏，你需要一个地方集中看这些错误，而不是用户来告诉你。

五、企业化升级的底层逻辑图

1. 不只是“换框架”，而是升级四个层次

```
第一层：页面层升级
原生 HTML -> 组件化前端

第二层：数据层升级
CSV / JS -> PostgreSQL

第三层：系统层升级
单脚本原型 -> 模块化前后端工程
```

```
第四层：治理层升级
手工维护 -> 权限、日志、测试、监控、自动部署
```

2. 你的项目升级思路图

```
当前项目已有：
业务逻辑完整
  ├─ 群体画像
  ├─ 个人画像
  ├─ 高校对标
  ├─ 资源承接
  ├─ 方法说明
  └─ AI 助手

升级时不要丢掉业务逻辑
  -> 保留页面含义
  -> 保留后端业务处理
  -> 保留数据口径
  -> 升级前端框架
  -> 升级数据存储
  -> 增加企业能力
```

3. 你以后要具备的完整工程图

```
用户
  -> 前端 Next.js
       -> 页面组件
       -> 图表组件
       -> 权限路由
       -> 管理后台
  -> 调用 FastAPI
       -> 画像接口
       -> 对标接口
       -> AI 接口
       -> 权限接口
  -> 访问 PostgreSQL
       -> 学生数据
       -> 对标数据
       -> 资源数据
       -> 用户与日志
  -> GitHub Actions 自动测试与部署
  -> Sentry 负责错误监控
```

六、八周总体安排总览

先看全局，再看每周细节。

八周总目标图

第1–2周：技术选型和前端重构起步
第3–4周：后端模块化和数据库落地
第5–6周：核心页面迁移与数据正式化
第7周：权限、测试、监控
第8周：上线收尾、文档、复盘、下一阶段规划

八周完成后的目标状态

8 周结束时，理想成果不是“所有功能全部完美”，而是：

- 新前端项目已经搭起来
- 核心页面至少迁移了 2 到 4 个
- 后端不再是单文件原型结构
- 数据库已经能承载核心数据
- 学生数据和对标数据开始接口化
- 具备基础登录 / 权限骨架
- 有最基础的测试和监控
- 有可继续长期开发的项目结构

七、八周详细计划

下面是最核心的部分。

第 1 周：定路线、搭脚手架、补核心基础

本周目标

- 明确最终技术路线
- 初始化新的前端工程
- 开始系统学习 React / Next.js / TypeScript 基础
- 梳理旧项目页面地图和接口地图

为什么先做这个

因为企业化升级最怕“边学边乱改”。第一周必须先把路线定住，不然第 3 周很可能又推翻重来。

本周学习内容

1. React 基础

要学：

- 组件是什么
- `props`

- `state`
- `useEffect`
- 事件处理
- 条件渲染
- 列表渲染

你怎么用：

- 把现在的 KPI 卡、导航、学生卡片都看成组件

2. Next.js 基础

要学：

- `app` 路由模式
- 页面文件结构
- 布局文件
- 客户端组件和服务端组件的概念

你怎么用：

- 把 `index_formal.html` 理解成未来一个 `page.tsx`
- 把公共导航理解成 `layout.tsx` 或公共组件

3. TypeScript 基础

要学：

- 基础类型
- 接口 `interface`
- 类型别名 `type`
- 函数参数类型
- API 返回类型

你怎么用：

- 给学生对象、风险对象、对标项目对象定义类型

本周要做的项目任务

- 新建 `frontend/` 项目
- 使用 Next.js + TypeScript 初始化
- 建立基础目录：
  - `app/`
  - `components/`
  - `types/`
  - `services/`

    - `styles/`
- 把当前所有页面做一张“页面地图”
- 把现有 FastAPI 接口做一张“接口地图”
- 写一份迁移清单，列出哪些页面优先迁移

本周交付物

- 新前端项目能跑起来
- 一个简单首页能打开
- 页面与接口清单文档
- 核心类型定义初稿

本周验收标准

- 能在本地打开新的 Next.js 页面
- 你能清楚说出：页面、组件、接口、数据四者分别是什么

第 2 周：组件化前端基础与设计系统初版

本周目标

- 建立公共布局和组件系统
- 统一视觉规范
- 学会前端如何请求后端数据

本周学习内容

1. 组件设计思维

要学：

- 页面拆分原则
- 容器组件和展示组件
- 组件复用
- 组件命名规范

你怎么用：

- 以后不是每个页面都重新写按钮、卡片、标签、表格

2. 数据请求

要学：

- 前端如何调用后端 API
- 请求层和页面层分离
- loading / error / empty 状态处理

你怎么用：

- 调用现在 FastAPI 的 `/api/formal/overview`
- 先让新前端拿到旧后端的数据

本周要做的项目任务

- 搭好全站 Layout
- 抽离基础组件：
  - `PageShell`
  - `SectionCard`
  - `KpiCard`
  - `Tag`
  - `EmptyState`
  - `LoadingBlock`
  - `ErrorBlock`
- 建立主题变量：
  - 颜色
  - 间距
  - 标题层级
  - 阴影
  - 圆角
- 新建 `services/api.ts` 统一封装接口
- 把群体页顶部框架迁移到新前端里

本周交付物

- 可复用组件基础库
- 基础布局系统
- 前端接口请求层
- 群体页壳子版本

本周验收标准

- 群体页新前端可以调通至少一个接口
- 公共组件不再散落到每个页面里

第 3 周：后端模块化改造

本周目标

- 把后端从单文件原型逐步整理成模块化项目
- 学习 FastAPI 的工程化组织方式

本周学习内容

1. FastAPI 工程化结构

要学：

- 路由拆分
- Schema 拆分
- Service 层
- 配置管理
- 中间件

你怎么用：

- 把现在 `backend_formal.py` 里混在一起的逻辑拆开

2. Pydantic / Schema 设计

要学：

- 请求和响应模型
- 字段校验
- 返回结构统一

你怎么用：

- 给学生列表、学生详情、群体概览建立结构化响应模型

本周要做的项目任务

- 新建 `backend/app/`
- 拆分模块：
  - `api/routes/`
  - `schemas/`
  - `services/`
  - `core/`
- 把以下接口先迁过去：
  - `overview`
  - `students`
  - `student/{id}`
  - `warnings`
- 统一响应格式
- 把配置项从代码中剥离出来

本周交付物

- 模块化后的后端骨架
- 至少 4 个核心接口可正常使用
- 统一配置方案

本周验收标准

- 后端仍然能跑
- 核心接口地址不乱
- 代码目录比现在清晰很多

第 4 周：数据库设计与数据入库

本周目标

- 正式引入 PostgreSQL
- 学习 SQLAlchemy 和 Alembic
- 把 CSV / JS 数据逐步导入数据库

本周学习内容

1. PostgreSQL 基础

要学：

- 表、字段、主键、索引
- 一对多关系
- 基础查询

你怎么用：

- 设计学生表、维度表、对标项目表

2. SQLAlchemy

要学：

- 模型定义
- Session
- 查询
- 插入更新

3. Alembic

要学：

- 初始化迁移
- 生成迁移脚本
- 执行迁移

本周要做的项目任务

- 启动 PostgreSQL
- 定义核心表：
  - `students`
  - `student_dimensions`
  - `student_risks`
  - `student_advice`
  - `benchmark_projects`
  - `resource_items`
- 建入库脚本：
  - 从 `formal_student_data.csv` 导入
  - 从 `benchmark_data.js` 转成可导入数据
- 为后端写数据库访问层

本周交付物

- 数据库初版
- 数据表结构图
- 导入脚本
- Alembic 迁移记录

本周验收标准

- 至少能从数据库查出学生列表和对标项目列表
- 新后端已经不再只能依赖 CSV

第 5 周：迁移群体页与学生页

本周目标

- 把最核心的两个页面迁移到新前端
- 用正式接口和数据库支持页面

本周学习内容

1. 图表组件封装

要学：

- ECharts 在 React 中怎么封装
- 图表数据和页面逻辑怎么分开

2. 页面状态管理

要学：

- 筛选条件管理
- 图表点击联动
- 弹窗状态管理

本周要做的项目任务

- 迁移群体页：
  - KPI 区
  - 维度图
  - 类型分布
  - 预警分布
  - 学生列表
- 迁移学生页：
  - 基本档案
  - 雷达图
  - 风险标签
  - 建议路径
- 把老页面中重复逻辑组件化

本周交付物

- 新版群体页
- 新版学生页
- ECharts 组件封装

本周验收标准

- 用户可以在新前端中完成“看群体 -> 点学生 -> 看个人画像”的主链路

第 6 周：迁移对标模块与资源模块

本周目标

- 完成对标模块迁移
- 让对标数据从正式接口读取

- 开始准备后台维护化

本周学习内容

1. 表格与筛选

要学：

- 列表页筛选
- 表格渲染
- 排序
- 分页思维

2. 详情页与比较页组织

要学：

- URL 参数驱动页面
- 详情页和比较页的数据组织方式

本周要做的项目任务

- 迁移：
  - 对标总览页
  - 对标详情页
  - 多校比较页
  - 资源导航页
- 把 `benchmark_data.js` 对标数据改成后端接口读取
- 后端补充对标项目 CRUD 基础能力

本周交付物

- 新版对标模块
- 对标接口
- 资源页迁移版

本周验收标准

- 用户可以在新前端完成“看总览 -> 看详情 -> 做比较”的完整链路

第 7 周：权限、测试、监控与质量保障

本周目标

- 补企业级系统最关键的基础治理能力

本周学习内容

1. 登录与权限

要学：

- 用户、角色、权限的概念
- 路由保护
- 登录态

2. 测试

要学：

- 后端接口测试
- 前端端到端测试

3. 监控

要学：

- 页面错误监控
- 接口异常监控

本周要做的项目任务

- 添加基础登录页
- 区分至少两种角色：
  - 管理员
  - 查看者
- 用 Playwright 写 3 到 5 条核心用例：
  - 打开首页
  - 查看学生详情
  - 多校比较
  - 登录退出
- 接入 Sentry
- GitHub Actions 增加自动测试

本周交付物

- 登录骨架
- 基础权限控制
- 自动化测试初版

- 错误监控初版

本周验收标准

- 至少关键页面有基础权限保护
- 测试能自动跑
- 线上异常能被记录

第 8 周：收尾上线、文档沉淀、下一阶段规划

本周目标

- 把整个升级结果整理为一个“可继续迭代”的正式工程

本周学习内容

1. Docker 基础

要学：

- Dockerfile
- 多服务运行思路

2. 部署与环境管理

要学：

- 开发环境
- 测试环境
- 生产环境
- 环境变量管理

本周要做的项目任务

- 给前后端写 Docker 配置
- 梳理部署文档
- 输出项目 README 和开发文档
- 梳理目录结构
- 写下一阶段迭代路线：
    - 后台内容维护
    - 数据审核
    - 导出报告
    - 更细粒度权限

本周交付物

- 可部署工程
- 完整说明文档
- 目录结构稳定版
- 二阶段路线图

本周验收标准

- 新系统具备长期开发基础
- 别人接手时不会一脸茫然

八、学习顺序建议

很多人容易犯的错误是同时学太多工具。更合理的顺序是：

```
先学前端框架基础
  -> 再学组件与类型
  -> 再学后端工程化
  -> 再学数据库
  -> 再学鉴权与测试
  -> 最后补 Docker / 监控 / 自动化
```

推荐优先级

第一优先级

- React
- Next.js
- TypeScript
- FastAPI 工程化

第二优先级

- PostgreSQL
- SQLAlchemy
- Alembic

第三优先级

- Auth.js / 登录权限
- Playwright
- GitHub Actions

第四优先级

- Docker
- Sentry
- 性能优化

九、你后面最需要形成的能力，不是背 API，而是这几种思维

1. 组件思维

页面不是一整块，而是一组可复用单元。

2. 数据思维

显示出来的页面背后应该有明确的数据结构和数据来源。

3. 分层思维

前端、后端、数据库、部署、监控，不要混在一起。

4. 可维护思维

不是“能跑就行”，而是“过三个月还能改”。

5. 系统思维

企业系统不是页面集合，而是：

- 权限
- 数据
- 流程
- 审计
- 稳定性

共同构成的整体。

十、最容易踩的坑

坑 1：一上来就全量重写

后果：

- 业务逻辑丢失
- 页面细节丢失
- 做了很久反而不如现在能用

坑 2：只换前端，不动数据

后果：

- 看起来更现代
- 本质还是 demo

坑 3：同时学太多技术

后果：

- 每个都懂一点
- 没一个真正落地

坑 4：太早追求复杂架构

比如一开始就搞：

- 微服务
- 多前端仓库
- 一堆中间件

对你现在阶段没有必要。

坑 5：忽略文档和测试

企业化不是代码多，而是项目更稳。

十一、八周之外的中长期规划

如果 8 周升级做完，下一阶段可以继续做这些：

1. 后台内容维护系统

让管理员能维护：

- 对标项目
- 资源导航
- 政策说明

2. 数据审核机制

给每条数据增加：

- 来源

- 更新时间
- 审核状态
- 审核人

3. 报告导出

支持：

- 个人画像报告导出
- 群体统计报告导出
- 对标对比报告导出

4. 更正式的权限系统

- 管理员
- 导师
- 院系管理者
- 只读访客

5. 更稳定的生产部署

- 专属域名
- HTTPS
- 数据备份
- 多环境发布

十二、给你的最终建议

如果你只记住一句话，那就是：

你后面要做的不是“从 HTML 换到 React”，而是把“高质量原型”升级成“可维护的正式系统”。

所以真正的升级顺序应该是：

```
保留业务逻辑
  -> 升级前端工程
  -> 升级后端结构
  -> 升级数据层
  -> 补权限、测试、监控、部署
```

而不是：

```
因为 React 热门，所以全部推倒重写
```

这是两个完全不同的思路。

前者大概率能成功，后者大概率会让项目中途失控。

十三、推荐官方学习资料

下面这些官方资料，建议你在 8 周中按需查阅。

前端与框架

- React 官方： https://react.dev/learn
- React 官方：Start a New React Project https://react.dev/learn/start-a-new-react-project
- Next.js 官方：App Router Getting Started https://nextjs.org/docs/app/getting-started
- Vue 官方： https://vuejs.org/guide/introduction.html
- Nuxt 官方： https://nuxt.com/docs

后端与数据库

- FastAPI 官方： https://fastapi.tiangolo.com/tutorial/
- SQLAlchemy 官方 ORM Quick Start： https://docs.sqlalchemy.org/20/orm/quickstart.html
- Alembic 官方： https://alembic.sqlalchemy.org/en/latest/tutorial.html
- PostgreSQL 官方文档： https://www.postgresql.org/docs/current/tutorial-start.html

工程与质量

- Playwright 官方： https://playwright.dev/docs/intro
- GitHub Actions 官方： https://docs.github.com/actions/quickstart
- Docker 官方： https://docs.docker.com/get-started/
- Sentry Next.js 官方： https://docs.sentry.io/platforms/javascript/guides/nextjs/

鉴权

- Auth.js 官方： https://authjs.dev/getting-started

十四、最后的落地建议

如果你接下来立刻开始做，我建议你的第一动作不是学一堆视频，而是做下面三件事：

1. 在项目旁边新建一个 `frontend/`，用 Next.js + TypeScript 初始化

2. 把你现在的网站页面和接口先画成一张迁移清单
3. 先只迁移两个最核心页面：
    - 群体页
    - 个人画像页

只要这两页迁成功，整个企业化升级路线就已经真正启动了。

附录：一句话版本的技术定位

当前项目

高质量可上线原型系统

目标项目

可持续维护的正式数据驱动平台

核心升级路线

```
HTML 原型 -> 组件化前端
CSV/JS 数据 -> PostgreSQL
单文件脚本 -> 模块化工程
手动维护 -> 自动化与治理能力
```